计算机系统 I：多周期处理器设计 (Multi-Cycle Processor Design) 复习讲义

目录

1 为什么需要多周期？(Why Multi-Cycle?) 2
1.1 提升程序性能的三个方向 . . . 2
1.2 单周期 vs. 多周期对比 . . . 2
1.3 多周期 CPU 的基本设计原则 . . . 2

2 多周期数据通路 (Multi-Cycle Datapath) 3
2.1 数据通路总体变化 . . . 3
2.2 新增的临时寄存器 (New Temporary Registers) . . . 3
2.3 控制信号汇总 (Control Signals Summary) . . . 3

3 指令执行的各阶段 (Instruction Execution Stages) 5
3.1 阶段一：取指 (IF, Instruction Fetch) . . . 5
3.2 阶段二：译码与读取操作数 (ID, Instruction Decode & Register Fetch) . . . 5
3.3 阶段三：执行 (EX, Execute) . . . 5
3.4 阶段四：访存 (MEM, Memory Access) . . . 6
3.5 阶段五：写回 (WB, Write Back) . . . 6
3.6 各类指令的阶段数汇总 . . . 6

4 控制器设计 (Controller Design) 7
4.1 控制器设计的分析框架 . . . 7
4.2 为什么单周期的控制器设计方式不适用? . . . 7
4.3 两种控制器实现方案 . . . 8

5 硬连线控制器：有限状态机 (Hardwired Controller: FSM) 8
5.1 FSM 设计思路 . . . 8
5.2 各状态定义与转移 . . . 8
5.3 硬连线控制器的电路实现 . . . 10

6 微程序控制器 (Micro-Program Controller) 11
6.1 历史背景 11
6.2 基本原理与术语 11
6.3 微程序控制器的实现架构 11
6.4 微程序控制器的两个核心问题 12
6.5 优缺点总结 12

7 多周期 CPU 的性能分析 (Performance Analysis) 12
7.1 CPI 计算 12
7.2 关键路径与时钟周期 13
7.3 性能对比计算 13
7.4 多周期设计的意义 14

8 全章总结 (Summary) 14

1 为什么需要多周期？(Why Multi-Cycle?)

1.1 提升程序性能的三个方向

提升程序运行速度，从体系结构角度主要有三个方向：

- **减少指令数 (Reduce Instruction Count)**：设计功能更强的指令（CISC 思路），或使用更优的编译器。
- **减少每条指令所需的时钟周期 (Reduce CPI)**：RISC 指令简单化，配合多周期、流水线等技术；引入多个执行单元（多发射）。
- **提高时钟频率 (Increase Clock Rate)**：采用更新工艺，优化关键路径，或引入流水线。

多周期设计属于第二个方向——通过允许简单指令更快完成来降低平均 CPI。

1.2 单周期 vs. 多周期对比

	单周期 (Single-Cycle)	**多周期 (Multi-Cycle)**
时钟周期长度	由最慢指令决定 (lw=800ps)	较短，由最复杂的单步操作决定
简单指令速度	慢（被迫等待最长时间）	快（简单指令占用更少周期）
硬件复用	差（ALU 和存储器各需两份）	好（一个 ALU、一块存储器可复用）
时序开销	每条指令只付一次	每个阶段都需要付出寄存器开销

单周期与多周期的根本区别

- **单周期**：一个时钟周期执行一**条完整指令**。
- **多周期**：一个时钟周期执行**指令执行的一个阶段**。

设计流程相同：都要设计数据通路 (Datapath) 和控制器 (Controller)。

1.3 多周期 CPU 的基本设计原则

1. 将指令执行拆分为若干**时间近似相等**的阶段，每个阶段占用一个时钟周期。
2. 每个时钟周期内最多完成一次存储器访问、一次寄存器读写，或一次 ALU 运算。
3. 上一个周期的执行结果必须**暂存在专用时序寄存器**中，供下一周期使用。
4. 时钟周期长度由**最复杂的单步操作**决定（而非最长指令）。

2 多周期数据通路 (Multi-Cycle Datapath)

2.1 数据通路总体变化

与单周期 CPU 相比，多周期 CPU 的数据通路作了以下关键调整：

- **存储器合并**：只保留一块统一的存储器（指令和数据共用），这也正是冯 · 诺依曼结构的本来样子。单周期 CPU 需要将其拆分是因为一个周期内可能同时读指令和读/写数据；多周期 CPU 由于分阶段执行，不会产生冲突。
- **ALU 合并**：只保留一个 ALU，不再需要单独的地址加法器，通过分时复用完成不同阶段的计算。
- **新增临时寄存器**：为了将执行结果从一个周期传递到下一个周期，需要引入多个临时寄存器（下节详述）。

2.2 新增的临时寄存器 (New Temporary Registers)

多周期 CPU 中新增的 5 个临时寄存器

- **IR（指令寄存器，Instruction Register）**：取指阶段结束后，将从存储器取出的指令锁存于此，供后续译码阶段使用。
- **MDR（内存数据寄存器，Memory Data Register）**：访存阶段结束后，将从内存读出的数据暂存于此，供写回阶段使用。
- **A**：译码/取操作数阶段结束后，暂存从寄存器文件读出的 rs1 数据。
- **B**：译码/取操作数阶段结束后，暂存从寄存器文件读出的 rs2 数据。
- **ALUout**：执行阶段结束后，暂存 ALU 的计算结果，供访存或写回阶段使用。

核心思想：这些临时寄存器的作用是将指令执行在物理上划分为独立的阶段。每个阶段结束时，将结果锁入对应的临时寄存器，下一阶段从该寄存器读取，从而实现阶段间的” 接力”。

2.3 控制信号汇总 (Control Signals Summary)

多周期 CPU 中的控制信号比单周期更多，且同一信号在不同阶段的含义/取值可能不同。

1 位控制信号：

信号	为 0 时	为 1 时
RegDst	写入寄存器来自 rt 字段	写入寄存器来自 rd 字段
RegWrite	无操作	寄存器文件被写入
ALUSrcA	ALU 第一个操作数为 PC	ALU 第一个操作数为寄存器 A
MemRead	无操作	将 Address 处的内存数据送至内存数据输出
MemWrite	无操作	将写数据写入 Address 处内存
MemToReg	写回寄存器的值来自 ALUout	写回寄存器的值来自 MDR
IorD	PC 作为存储器地址	ALUout 作为存储器地址
IRWrite	无操作	将内存输出写入 IR
PCWrite	无操作	PC 被写入（来源由 PCSrc 决定）
PCWriteCond	无操作	若 ALU 的 Zero 信号有效，则写入 PC

2 位控制信号：

信号	值	含义
ALUOp	00	ALU 执行加法
	01	ALU 执行减法
	10	由 funct 字段决定操作
ALUSrcB	00	ALU B 端来自寄存器 B
	01	ALU B 端固定为 4（用于 PC+4）
	10	符号扩展后的立即数
	11	符号扩展后的立即数左移 2 位（分支目标偏移）
PCSrc	00	PC ← ALU 输出（PC+4，顺序执行）
	01	PC ← ALUout（分支目标地址）
	10	PC ← 跳转目标地址

3 指令执行的各阶段 (Instruction Execution Stages)

3.1 阶段一：取指 (IF, Instruction Fetch)

操作步骤：

1. 将 PC 的值作为地址送至存储器。
2. 从存储器中读取指令。
3. 将指令写入 IR（IRWrite=1），供下一周期使用。
4. 同时利用 ALU 计算 PC+4（ALUSrcA=0，ALUSrcB=01，ALUOp=00），并更新 PC（PCWrite=1，PCSrc=00）。

本周期控制信号：IorD=0，ALUSrcA=0，ALUSrcB=01，ALUOp=00，PCSrc=00，IRWrite=1，PCWrite=1，MemRead=1，MemWrite=0，RegWrite=0。

3.2 阶段二：译码与读取操作数 (ID, Instruction Decode & Register Fetch)

操作步骤：

1. 对 IR 中的指令进行译码。
2. **乐观地读取寄存器操作数**：无论指令是否实际使用 rs1 和 rs2，都将其读出并存入寄存器 A 和 B。（因为读操作无副作用，乐观读取可节省时间。）
3. **乐观地计算分支目标地址**：ALU 计算 PC + (符号扩展立即数 × 4)（ALUSrcA=0，ALUSrcB=11，ALUOp=00）。此时我们还不知道是否是分支指令，但" 反正现在没别的事做"，提前计算以备用。

本周期控制信号：ALUSrcA=0，ALUSrcB=11，ALUOp=00。

3.3 阶段三：执行 (EX, Execute)

各类指令在此阶段出现分叉：

指令类型	执行操作
访存指令 (Load/Store)	ALU 计算内存地址 = A + 符号扩展立即数（ALUSrcA=1, ALUSrcB=10, ALUOp=00), 结果存入 ALUout
R 型指令	ALU 根据 funct 字段执行对应运算（A op B), ALUSrcA=1，ALUSrcB=00，ALUOp=10
分支指令 (beq)	ALU 计算 A-B（ALUSrcA=1，ALUSrcB=00，ALUOp=01)，产生 Zero 信号；PC 更新使用 ID 阶段预算的分支目标
跳转指令 (J)	直接产生跳转目标地址，PCSrc=10，PCWrite=1

3.4 阶段四：访存 (MEM, Memory Access)

仅 Load 和 Store 指令使用此阶段：

- **Load**：以 ALUout 中的地址从存储器读数据（IorD=1，MemRead=1)，结果存入 MDR。
- **Store**：以 ALUout 中的地址向存储器写入 B 寄存器中的数据（IorD=1，MemWrite=1)。Store 指令在此阶段结束后即完成，不需要写回阶段。

其他指令（R 型、分支、跳转）**跳过此阶段**，直接进入写回。

3.5 阶段五：写回 (WB, Write Back)

- **Load 指令**：将 MDR 中的数据写入目标寄存器(RegDst=0, MemToReg=1, RegWrite=1)。
- **R 型指令**：将 ALUout 中的结果写入 rd（RegDst=1，MemToReg=0，RegWrite=1)。
- **I 型算术指令（如 addi)**：将 ALUout 中的结果写入 rt（RegDst=0，MemToReg=0，RegWrite=1)。

3.6 各类指令的阶段数汇总

指令类型	所需阶段数	阶段序列
beq, j（跳转/分支）	3	IF → ID → EX
R 型, Store (sw/sd)	4	IF → ID → EX → WB / IF → ID → EX → MEM
Load (lw/ld)	5	IF → ID → EX → MEM → WB

4 控制器设计 (Controller Design)

4.1 控制器设计的分析框架

控制器的设计从**因果分析**入手，核心思路与单周期 CPU 一致，但需考虑阶段信息：

- **信息来源**：指令的 32 位机器码（opcode、funct3、funct7、rd、rs1、rs2 等字段）以及当前所处的执行阶段。
- **控制目标一：顺序部件的操作选择**（寄存器读/写使能、存储器读/写使能等）。
- **控制目标二：数据流控制**（多路选择器的选择信号，决定数据从哪里来、到哪里去）。
- **控制目标三：ALU 操作选择**（根据指令类型和 funct 字段确定 ALU 执行何种运算）。
- **控制目标四：分支/跳转指令的 PC 更新**（决定是否以及如何更新 PC）。

与单周期 CPU 的关键区别：在多周期 CPU 中，同一控制信号在不同周期（不同阶段）的取值可能完全不同，因此控制器必须**感知当前阶段**，而不能像单周期那样只看 opcode 就给出固定输出。

4.2 为什么单周期的控制器设计方式不适用？

在单周期 CPU 中，每条指令的每个控制信号在整个时钟周期内**保持不变**。但在多周期 CPU 中，**同一个控制信号在不同周期的取值可能完全不同**。

示例：执行 `add x10, x12, x14`：

控制信号	周期 1 (IF)	周期 2 (ID)	周期 3 (EX)	周期 4 (WB)
PCWrite	1	0	0	0
IorD	0	–	–	–
MemRead	1	0	0	0
IRWrite	1	0	0	0
PCSrc	00	–	–	–
ALUOp	00	00	**10**	–
ALUSrcB	01	11	**00**	–
ALUSrcA	0	0	**1**	–
RegWrite	0	0	0	**1**

可以看出，ALUOp 在周期 2 (ID) 是 00，在周期 3 (EX) 变为 10——**控制信号随执行阶段变化**，这是多周期 CPU 控制器设计的核心挑战。

多周期控制器的关键特征

多周期 CPU 的控制器必须是**时序逻辑**（有状态的），而非单周期 CPU 的纯组合逻辑。控制器需要记住当前处于哪个执行阶段，并据此产生正确的控制信号。

4.3 两种控制器实现方案

	硬连线控制器 (Hardwired)	**微程序控制器 (Microprogram)**
实现方式	有限状态机 (FSM) + PLA 电路 + 状态寄存器	控制存储器 (Control Storage/ROM) + 微指令
适用场景	RISC（指令简单、数量少）	CISC（指令复杂、数量多）
优点	控制信号产生延迟短，性能高	设计灵活，易于修改和扩展 ISA
缺点	ISA 复杂时难以维护和更新	执行时间比硬连线长，效率较低

5 硬连线控制器：有限状态机 (Hardwired Controller: FSM)

5.1 FSM 设计思路

硬连线控制器的核心是**有限状态机 (Finite State Machine, FSM)**：

- 每个状态 (State) 对应指令执行的某一个阶段，并确定该阶段的所有控制信号取值。
- 状态之间的**转移条件**通常依赖指令的 opcode 字段（从 IR 读取）。
- 电路实现：PLA（可编程逻辑阵列）实现组合逻辑，状态寄存器存储当前状态，每个时钟上升沿更新状态。

5.2 各状态定义与转移

以下为完整 FSM 的状态列表（状态编号 S0–S11）：

FSM 状态图（所有指令类型）

S0：取指 (IF)（复位后进入）

- 控制：IorD=0, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, IRWrite=1, PCWrite=1
- 无条件转移至 S1

S1：译码/读操作数 (ID)

- 控制：ALUSrcA=0，ALUSrcB=11，ALUOp=00（乐观计算分支目标）
- 根据 opcode 分叉：
  - Op=LW 或 Op=SW → S2（内存地址计算）
  - Op=R-type → S6（R 型执行）
  - Op=beq → S8（分支执行）
  - Op=addi → S9（addi 执行）
  - Op=J → S11（跳转）

S2：内存地址计算 (MemAdr)（lw 和 sw 共用）

- 控制：ALUSrcA=1，ALUSrcB=10，ALUOp=00
- Op=LW → S3；Op=SW → S5

S3：内存读取 (MemRead)（lw）

- 控制：IorD=1，MemRead=1
- 无条件转移至 S4

S4：Load 写回 (MemWriteback)（lw）

- 控制：RegDst=0，MemToReg=1，RegWrite=1
- 无条件转移至 S0

S5：内存写入 (MemWrite)（sw）

- 控制：IorD=1，MemWrite=1
- 无条件转移至 S0

S6：R 型执行 (Execute)

- 控制：ALUSrcA=1，ALUSrcB=00，ALUOp=10
- 无条件转移至 S7

S7：R 型写回 (ALUWriteback)

- 控制：RegDst=1，MemToReg=0，RegWrite=1
- 无条件转移至 S0

S8：分支执行 (Branch)（beq）

- 控制：ALUSrcA=1，ALUSrcB=00，ALUOp=01，PCSrc=01，PCWriteCond=1
- 无条件转移至 S0

S9：addi 执行 (AddiExecute)

- 控制：ALUSrcA=1，ALUSrcB=10，ALUOp=00
- 无条件转移至 S10

S10：addi 写回 (AddiWriteback)

- 控制：RegDst=0，MemToReg=0，RegWrite=1
- 无条件转移至 S0

S11：跳转 (Jump)

- 控制：PCSrc=10，PCWrite=1
- 无条件转移至 S0

5.3 硬连线控制器的电路实现

硬连线控制器电路结构

硬连线控制器由以下三部分构成：

- **状态寄存器 (State Register)**：存储 FSM 的当前状态（如 S0–S11），每个时钟上升沿根据组合逻辑输出更新状态。
- **PLA（可编程逻辑阵列，Programmable Logic Array)**：实现组合逻辑部分，包括两个子功能：
  - **输出逻辑**：根据当前状态产生本周期的所有控制信号（如 ALUSrcA、ALUOp 等）。
  - **次态逻辑**：根据当前状态 + 指令的 opcode 字段，计算下一个状态。
- **接口信号**：控制器接收数据通路的 opcode（来自 IR）和时钟信号，输出各种控制信号到数据通路。

工作流程：时钟上升沿 → 状态寄存器锁存新状态 → PLA 根据新状态和 opcode 输出控制信号 → 数据通路执行对应操作 → 等待下一个时钟沿。

6 微程序控制器 (Micro-Program Controller)

6.1 历史背景

- **1951 年**：M. V. Wilkes 提出微程序控制器的概念。
- **1964 年**：IBM 360 系列首次在商用机器上部署微程序控制器。
- **1970 年代起**：随着 VLSI 技术发展得到广泛应用。
- **现状**：大多数 CISC 处理器（如 x86）均采用微程序控制器。

6.2 基本原理与术语

微程序控制器核心术语

- **微程序 (Micro-program)**：由一系列微指令构成的序列，每条 ISA 指令对应一个微程序。
- **微指令 (Micro-instruction)**：一个 0/1 序列，由若干微命令组成；也称**控制字 (Control Word, CW)**。每条微指令对应 FSM 中的一个状态。
- **微命令 (Micro-command)**：控制部件发送给执行部件的控制信号（如寄存器的时钟使能信号、MUX 的选择信号）。
- **微操作 (Micro-operation)**：与微命令一一对应，是执行部件响应微命令所完成的最基本操作（类似数据通路中控制信号与数据流操作的关系）。
- **控制存储器 (Control Storage, CS)**：一块 ROM，存储所有微程序（即所有微指令的集合）。
- **微程序计数器 (μPC)**：类似 ISA 级别的 PC，指向控制存储器中下一条微指令的地址。

6.3 微程序控制器的实现架构

微程序控制器的主要组成部件：

- **μPC（微程序计数器）**：指向 CS 中当前执行的微指令地址。
- **控制存储器 (CS/ROM)**：存储全部微指令，通过 μPC 寻址。
- **微指令寄存器 (μIR)**：暂存从 CS 中取出的当前微指令。
- **起始/分支地址生成器**：根据 IR 中的 opcode 确定每条 ISA 指令对应微程序的起始地址，以及微指令内部的分支目标。
- **分支控制逻辑**：根据条件码 (Condition Code) 决定是否转移到分支微指令地址。

6.4 微程序控制器的两个核心问题

1. **微指令格式与微命令编码**：如何在有限的位宽内编码所有微命令？
   - **直接编码**（水平型微指令）：每一位直接对应一个微命令，控制灵活但指令字很长。
   - **间接编码**（垂直型微指令）：对微命令分组编码，指令字短但需要额外译码延迟。
2. **下一微指令地址的确定**：通常有两种方式：
   - **μPC 顺序加一**：类似普通程序计数器的顺序递增。
   - **微指令中显式存储下一地址**：微指令格式中包含跳转地址字段，支持灵活转移。

6.5 优缺点总结

微程序控制器的优缺点

优点：

- 显著降低控制器设计难度，尤其是对于复杂指令集 (CISC)。
- 灵活性高：修改微程序即可调整 ISA，无需重新布线。

缺点：

- 执行速度慢于硬连线控制器（访问 CS 的延迟）。
- 效率相对较低（引入额外存储和译码开销）。

7 多周期 CPU 的性能分析 (Performance Analysis)

7.1 CPI 计算

多周期 CPU 中，不同指令的 CPI 不同，需要用**加权平均 CPI** 来评估整体性能。
各类指令所需周期数：

指令类型	所需周期数
分支 (beq)、跳转 (j)	3
R 型、Store (sw)	4
Load (lw)	5

假设典型程序的指令分布：

- Load：25%
- Store：10%

- Branch：11%
- Jump：2%
- R 型：52%

计算加权平均 CPI：

$$\begin{aligned}\overline{\text{CPI}} &= (0.11 + 0.02) \times 3 + (0.52 + 0.10) \times 4 + 0.25 \times 5 \\ &= 0.13 \times 3 + 0.62 \times 4 + 0.25 \times 5 \\ &= 0.39 + 2.48 + 1.25 \\ &= \mathbf{4.12}\end{aligned}$$

7.2 关键路径与时钟周期

多周期 CPU 的时钟周期由**单个阶段中最长的操作**决定。假设各部件延迟为：

部件	延迟
存储器读/写	最长，主导关键路径
ALU 运算	较长
寄存器读/写	较短

关键路径分析：在多周期设计中，存储器读操作（取指阶段读指令存储器，或访存阶段读数据存储器）通常是最慢的单步操作，**主导时钟周期**。

在本课程的具体参数下：$T_c = 325\,\text{ps}$。

7.3 性能对比计算

例题：以 1000 亿条指令的程序为例，分别比较单周期和多周期 CPU 的执行时间。

多周期 CPU：

$$T_{\text{multi}} = 10^{11} \times 4.12 \times 325\,\text{ps} = 10^{11} \times 4.12 \times 325 \times 10^{-12}\,\text{s} \approx \mathbf{133.9}\,\text{s}$$

单周期 CPU（$T_c = 925\,\text{ps}$，CPI=1）：

$$T_{\text{single}} = 10^{11} \times 1 \times 925\,\text{ps} = 10^{11} \times 925 \times 10^{-12}\,\text{s} = \mathbf{92.5}\,\text{s}$$

反直觉的结论：单周期反而更快?

在此例中，**单周期 CPU（92.5s）比多周期 CPU（133.9s）执行更快**。
原因分析：

- 多周期缩短了时钟周期（925ps → 325ps），但代价是 CPI 从 1 增加到 4.12。
- 整体执行时间 = IC × CPI × T_c：$1 \times 925 < 4.12 \times 325$（$925 < 1339$）。
- **时序开销 (Sequencing Overhead)** 被多次支付：每个阶段都需要额外的寄存器写入开销，累积后抵消了缩短时钟周期带来的收益。

7.4 多周期设计的意义

尽管多周期 CPU 在简单实现下并不比单周期快，但它提供了一个**重要的设计视角**：

多周期的核心贡献

多周期将指令执行分解为**细粒度的阶段**，这为**流水线 (Pipelining)** 技术奠定了基础。
流水线的思路：在多周期的基础上，让多条指令的不同阶段**重叠执行**（类似工厂流水线），从而在不降低每条指令延迟的前提下，大幅提升**吞吐量 (Throughput)**——这将是下一讲（系统 II）的核心主题。

8 全章总结 (Summary)

1. **多周期 CPU 的动机**：单周期 CPU 的时钟周期受最慢指令（lw，925ps）约束；多周期通过缩短时钟周期（每周期只完成一个阶段）并允许简单指令更快完成，改善平均 CPI。
2. **数据通路变化**：合并存储器（统一指令/数据存储器）和 ALU（去掉多余加法器）；新增 5 个临时寄存器（IR、MDR、A、B、ALUout）实现阶段间数据传递。
3. **五个执行阶段**：IF（取指）→ ID（译码 + 读操作数）→ EX（执行）→ MEM（访存，仅 lw/sw）→ WB（写回）。不同指令在 EX 之后走不同路径，周期数为 3–5。
4. **控制信号随阶段变化**：同一信号在不同周期取值不同，因此控制器必须是时序逻辑，需记录当前阶段。
5. **两种控制器实现**：
   - **硬连线控制器**：FSM（S0–S11）+ PLA，适合 RISC，性能高但难以修改。
   - **微程序控制器**：将每条 ISA 指令翻译为存储在 CS/ROM 中的微指令序列，适合 CISC，灵活但效率较低。

6. **性能分析**：典型程序的加权平均 CPI ≈4.12，$T_c = 325\,\text{ps}$，执行时间 ≈133.9s > 单周期 92.5s（$T_c = 925\,\text{ps}$）。多周期实现未能超越单周期，原因是时序开销多次支付抵消了缩短时钟周期的收益。

7. **设计启示**：多周期揭示了指令的细粒度执行结构，直接引出流水线技术——通过让多条指令的不同阶段重叠执行，在不增加硬件的情况下大幅提升吞吐量。

”多周期是单周期到流水线的桥梁：它分解了问题，流水线则让多个问题同步推进。”